Лекція 7. Nested / Inner classes, Enums

План:

1. **Нестатичні та статичні вкладені класи.**
2. **Анонімні внутрішні класи.**
3. **Тип даних enum.**
4. **Використання вкладених та внутрішніх класів.**

Якщо визначення класу розташовується у тілі іншого класу, то такий клас називається *вкладеним* (*inner class*). Зона видимості вкладеного класу обмежена зоною видимості зовнішнього класу. Таким чином, якщо один клас визначений усередині іншого, то це означає, що перший не може існувати незалежно від другого. Вкладений клас має доступ до членів класу в якому він описаний, навіть у випадку якщо вони оголошені із модифікатором доступу private. Проте зовнішній клас не має доступу до членів вкладеного класу. Вкладений клас, який оголошений безпосередньо усередині області видимості свого зовнішнього класу, є його членом.

Статичні вкладені класи

Статичний вкладений клас – це клас, який визначений із застосуванням модифікатору static. Статичний вкладений клас має доступ до нестатичних членів свого зовнішнього класу тільки через його об’єкт. Тобто він не може безпосередньо посилатися на нестатичні члени свого зовнішнього класу, через що статичні вкладені класи використовуються досить рідко. Розглянемо приклад реалізації статичного вкладеного класу.

```
// Використання статичного вкладеного класу
class Outer {
  public Outer() {

        // Створення екземпляру статичного вкладеного класу
        Inner inn = new Inner();
        inn.callInner();
  }

  private int outer_x;
  static int outer_y;

  // Визначення статичного вкладеного класу
  static class Inner {
        void callOuterX(int x) {
              Outer out = new Outer();
              out.outer_x = x;
              System.out.println("Значення outer_x: " +  out.outer_x);
        }
        void callOuterY(int y) {
              Outer.outer_y = y;
              System.out.println("Значення outer_y: " + Outer.outer_y);
              }
        void callInner() {
              System.out.println("Метод callInner() із класу Inner");
```

```
        }
    }
}

public class Main {
    public static void main (String args[]) {

        Outer out = new Outer();

        // Створення екземпляру статичного вкладеного класу
        Outer.Inner oi = new Outer.Inner();
        oi.callOuterX(5);
        oi.callOuterY(10);
    }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Метод callInner() із класу Inner
Метод callInner() із класу Inner
Значення outer_x: 5
Значення outer_y: 10
```

Нестатичні внутрішні класи

Нестатичний вкладений клас має доступ до усіх змінних і методів свого зовнішнього класу і може безпосередньо посилатися на них так само, як це роблять інші нестатичні члени зовнішнього класу. Досить часто нестатичний вкладений клас називають також внутрішнім класом. У наступній програмі демонструється робота із внутрішнім класом.

```
// Використання внутрішнього класу
class Outer {
    public Outer() {
        Inner inn = new Inner();
        inn.callInner();
    }

    private int outer_x;

    // Визначення внутрішнього класу
    class Inner {
        void callOuterX(int x) {
            outer_x = x;
            System.out.println("Значення outer_x: " +  outer_x);
        }
        void callInner() {
            System.out.println("Метод callInner() із класу Inner");
        }
    }
}

public class Main {
    public static void main (String args[]) {
```

```
        Outer out = new Outer();

        // Створення екземпляру внутрішнього класу
        Outer.Inner oi = out.new Inner();
        oi.callOuterX(5);
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Метод callInner() із класу Inner
Значення outer_x: 5
```

Як уже було сказано, внутрішній клас має доступ до усіх елементів свого зовнішнього класу, але не навпаки. Члени внутрішнього класу відомі тільки в його зоні видимості і не можуть бути використані зовнішнім класом. Також у внутрішньому класі заборонено оголошувати члени із модифікатором static та створювати перерахування.

Локальні внутрішні класи

Внутрішні класи можна визначати не тільки всередині інших класів, але і всередині зони видимості будь-якого блоку. Такі класи, визначені всередині блоку, називають локальними класами. Наприклад, локальний клас можна визначити в тілі методу, або всередині тіла циклу for, як показано в наступному прикладі.

```
// Використання статичного вкладеного класу
class Outer {

  private int outer_x;
  public Outer() {
        for(int i=0; i<5; i++) {

              // Визначення статичного вкладеного класу
              class Inner {
                    void callOuterX(int x) {
                          outer_x = x;
                          System.out.println("Значення outer_x: " +  outer_x);
                    }
              }
              Inner inn = new Inner();
              inn.callOuterX(i);
        }
  }
}

public class Main {
  public static void main (String args[]) {

        new Outer();
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Значення outer_x: 0
```

```
Значення outer_x: 1
Значення outer_x: 2
Значення outer_x: 3
Значення outer_x: 4
```

Такі класи зустрічаються досить рідко і мають велику кількість обмежень: 1) вони видимі тільки у межах блоку, в якому оголошені; 2) вони не можуть бути оголошені з модифікаторами private, public, protected або static; 3) вони не можуть мати статичних членів, окрім констант оголошених як static final.

Анонімні внутрішні класи

Працюючи з локальними внутрішніми класами, можна використати ще одну їх цікаву властивість. Якщо необхідно створити один єдиний екземпляр цього класу, то його ім'я можна опустити. Такий клас називається безіменним або анонімним внутрішнім класом.

Будь-які параметри, необхідні для створення об'єкту анонімного класу, задаються в круглих дужках, що йдуть за іменем супертипу. В загальному випадку синтаксис оголошення анонімного класу виглядає наступним чином:

```
new Супертип(список_параметрів)
{
  // тіло класу
};
```

Супертип – це ім'я інтерфейсу, який реалізує анонімний клас, або ім'я класу від якого він наслідує. Розглянемо програму, що демонструє використання анонімного класу.

```
// Використання анонімного класу
abstract class Outer {
  Outer() {
        System.out.println("Конструктор класу Outer");
  }
  abstract void call();
}

public class Main {

  Outer getOuter() {
        return

        // Початок анонімного класу
        new Outer() {
              {
                    System.out.println("Всередині анонімного класу");
              }
              void call() {
                    System.out.println("Метод call() із анонімного класу");
              }
        }; // Кінець анонімного класу
  }

  public static void main (String args[]) {
        Main m = new Main();
```

```
        Outer out = m.getOuter();
        out.call();
    }
}
```

Запустивши програму на виконання отримаємо наступний результат:

Конструктор класу Outer
Всередині анонімного класу
Метод call() із анонімного класу

Приведений вище код означає наступне: створюється новий об'єкт класу, що наслідує інтерфейс Outer, в якому у фігурних дужках { } визначений блок з оператором System.out.println() та перевизначається метод call(). В кінці закриваючої фігурної дужки анонімного класу проставляється оператор «;».

Перерахування на Java

Enum в Java - це коротка форма Enumeration. Клас Java Enum визначає перелік констант, значення яких ми не можемо змінити. Це схоже на кінцеву змінну. Ми завжди визначаємо константи Enum у верхньому регістрі.

Для оголошення enum ми повинні використовувати ключове слово enum, за яким слідує змінна. Всередині фігурних дужок ми оголошуємо константи, розділені комами.

```
enum Speed {
  SLOW, MEDIUM, FAST
}
```

Speed - представляє ім'я змінної Enum

Значення всередині фігурних дужок представляють константи Enum.

Особливості Enum

- Ми завжди повинні визначати константи переліку, використовуючи великі регістри
- Ми можемо оголосити перелік чисельності всередині або поза класом
- Enum - це клас, і кожна константа є об'єктом типу enum.
- Ми можемо оголосити поля та методи всередині переліку
- Кожна константа перерахування відокремлюється комою і повинна закінчуватися крапкою з комою, якщо перерахування містить також методи.

Доступ до констант Enum у Java

Ми можемо отримати доступ до констант enum двома різними способами:

- Створення змінної enum: ми можемо створити змінну enum, призначивши їй константу enum. Ми можемо використовувати цю змінну для друку значення.

```
enum Speed {
   SLOW, MEDIUM, FAST
  }
Speed s = Speed.MEDIUM;
System.out.println(d);
```

● Використання імені перерахування: ми можемо отримати безпосередній доступ до константи, використовуючи ім’я переліку.

```
enum Speed {
   SLOW, MEDIUM, FAST
  }
System.out.println(Speed.SLOW);
```

Використання основного методу всередині Enum в Java

Якщо ми використовуємо основний метод всередині Enum, ми можемо виконати Enum безпосередньо без будь-якого класу.

```
public enum Speed {
  SLOW, MEDIUM, FAST;

  public static void main(String[] args) {
    System.out.println(Speed.MEDIUM);
    System.out.println(Speed.FAST);
  }
}
Результат:
MEDIUM
FAST
```

Визначення Java Enum поза класом

Ми можемо оголосити перелічення поза класом, як ви можете бачити в наведеному нижче прикладі.

```
enum Speed {
  SLOW, MEDIUM, FAST
}
public class Sample {
  public static void main(String[] args) {
    Speed s = Speed.SLOW;
    System.out.println(d);
  }
}
Результат:
SLOW
```

Визначення Java Enum всередині класу

Нижче наведено приклад оголошення переліку всередині класу.

```
public class Sample {
  enum Speed {
    SLOW, MEDIUM, FAST
```

```
  }
  public static void main(String[] args) {
    Speed s = Speed.MEDIUM;
    System.out.println(d);
  }
}
```

Результат:

MEDIUM

Перерахувати за допомогою оператора if-else

Ми також можемо використовувати змінну enum у циклі - умовне твердження, як у наведеному нижче прикладі.

```
enum Speed {
  SLOW, MEDIUM, FAST
}
public class ConditionalEnum {
  public static void main(String[] args) {
    Speed s = Speed.MEDIUM;
    if(s == Speed.SLOW)
      System.out.println("Vehicle speed is slow");
    else if(s == Speed.MEDIUM)
      System.out.println("Vehicle speed is medium");
    else
      System.out.println("Vehicle speed is fast");
  }
}
```

Результат:

Vehicle speed is medium

Перерахувати в операторі switch

Нижче наведено приклад використання змінних enum всередині оператора switch шляхом передачі в якості аргументу.

```
enum Speed {
  SLOW, MEDIUM, FAST
}
public class ConditionalEnum {
  public static void main(String[] args) {
    Speed s = Speed.MEDIUM;

    switch(s) {
    case SLOW:
      System.out.println("Speed is Slow");
      break;
    case MEDIUM:
      System.out.println("Speed is Medium");
      break;
    case FAST:
      System.out.println("Speed is Fast");
      break;
```

```
    }
  }
}
```

Результат:
Speed is Medium

Ітерація значень перерахування за допомогою циклу For

Ми можемо перебирати значення перелічення в Java, перебирає константи перерахування в тому самому порядку, що і заявлені.

```
enum Speed {
  SLOW, MEDIUM, FAST
}
public class Sample{
  public static void main(String[] args) {
    for(Speed s : Speed.values())
      System.out.println(s);
  }
}
```

Результат:
SLOW
MEDIUM
FAST

Методи перерахування

Клас Java Enum відрізняється **методикою** для отримання значень.

values () - повертає всі константи, оголошені в переліку.

valueOf () - повертає константу переліку вказаного рядкового значення.

ordinal() - використовується для пошуку індексу константи перерахування, подібного до індексу масиву.

У наведеному нижче прикладі показано, як використовувати різні методи підтримки Java enum для отримання значень та їх індексу.

```
public class Test {
  enum Speed {
    SLOW, MEDIUM, FAST
  }
  public static void main(String[] args) {
    //Using values() method and ordinal method
    for(Speed s : Speed.values()) {
      System.out.println("Value: " + s + " Index: " + s.ordinal());
    }

    //Using valueOf() method
    System.out.println(Speed.valueOf("FAST"));
    //toString method
    Speed s = Speed.SLOW;
    System.out.println(s.toString());
  }
}
```

Результат:
Value: SLOW Index: 0

```
Value: MEDIUM Index: 1
Value: FAST Index: 2
FAST
SLOW
```

Ініціалізація значень константами перерахування

Ми можемо ініціалізувати значення констант Java-переліку, використовуючи конструктори. Перелік може містити конструктор, який викликається для кожної константи переліку. Конструктори перерахування за замовчуванням приватні. Нижче наведено приклад ініціалізації значень константами перечислення Java за допомогою конструкторів. Ми можемо отримати доступ до значень за допомогою приватного поля, яке ми ініціалізуємо в конструкторі.

```
public class Test {
  enum Speed {
    SLOW(15), MEDIUM(40), FAST(80);

    private int sp;
    private Speed(int sp) {
      this.sp = sp;
    }
  }
  public static void main(String[] args) {

    for(Speed s : Speed.values())
      System.out.println(s + ": " + s.sp);

  }
}
```

Результат:

```
SLOW: 15
MEDIUM: 40
FAST: 80
```

Перерахування та методи

Ми можемо визначити методи всередині перерахування. Всякий раз, коли перерахування у Java має як поля, так і методи, ми повинні закінчувати константи крапкою з комою. Ми можемо визначити звичайні та абстрактні методи всередині перерахування.

Нижче наведено приклад оголошення методів усередині перерахування.

```
public class EnumMethods {
  enum Speed {
    SLOW(10), MEDIUM(40), FAST(80);

    private int sp;

    Speed(int sp){
      this.sp = sp;
```

```
    }

    public int getSpeed() {
      return this.sp;
    }
  }
  public static void main(String[] args) {
    Speed s = Speed.SLOW;
    System.out.println(s + " :" + s.getSpeed());
    Speed s2 = Speed.FAST;
    System.out.println(s2 + " :" + s2.getSpeed());
  }
}
```

Результуат:

SLOW :10

FAST :80

Висновок:

Потужним інструментом мови Java є концепція вкладених та внутрішніх класів, основна сфера застосування, яких – це реалізація класів, які використовуються в контексті інших класів (наприклад, класи Map та Map.Entry), а також обробка подій.

Контрольні запитання:

1. Що таке статичний вкладений клас?
2. Що таке нестатичний внутрішній клас?
3. Що таке локальний внутрішній клас?
4. Що таке анонімний внутрішній клас?
5. Який синтаксис створення анонімного внутрішнього класу?
6. Який клас у мові Java знаходиться на вершині ієрархії?

Лекція 8. String and StringBuilder (StringBuffer) Classes
План:

1. **Клас String.**
2. **Створення та робота з рядками.**
3. **Пул об’єктів типу String.**
4. **Робота з класом StringBuilder.**
5. **Різниця між класами StringBuilder та StringBuffer.**

У мові Java рядок представляє собою набір символів, які упаковуються в об’єкт відповідного класу. Для роботи із рядками використовуються класи String, StringBuilder і StringBuffer із системної бібліотеки, які визначені в пакеті java.lang, що автоматично імпортується компілятором у всі програми. Ці класи оголошені із специфікатором final, що означає неможливість створення власних породжених класів з властивостями рядка.

***Клас* String**

Кожен рядок, що створюється за допомогою оператора new або за допомогою літерала (поміщеного в подвійні лапки), є об'єктом класу String. Особливістю об'єкта класу String є те, що його значення не може бути змінене після створення екземпляру класу, оскільки будь-яка зміна рядка призводить до створення нового об'єкту. При цьому посилання на об'єкт класу String можна змінити таким чином, щоб воно вказувало на інший об'єкт того ж самого класу.

У класі String визначено декілька конструкторів, які представлені у зведеній таблиці 8.1.

***Таблиця 8.1 – Конструктори, визначені у класі* String**

Конструктор	Опис
String()	Створює порожній рядок
String(String value)	Створює рядок з іншого рядка, у тому числі із рядкового літералу
String(char value[])	Створює рядок із масиву символів типу char
String(char value[], int offset, int count)	Створює масив з частини символьного масиву, починаючи із символу з номером offset і count символів після нього включно
String(byte value[])	Створює рядок із масиву символів Unicode
String(byte value[], int offset, int cout)	Створює рядок із частини масиву символів Unicode, починаючи із символу з номером offset і count символів після нього включно
String(StringBuffer value)	Створює рядок із об’єкта класу StringBuffer

String(StringBuilder value)	Створює рядок із об’єкта класу StringBuilder

Коли в програмі зустрічається літерал, поміщений в подвійні лапки, то автоматично створюється об'єкт типу String, на який можна встановити посилання. Таким чином, об'єкт класу String можна створити, присвоївши посиланню на клас значення існуючого літерала, або за допомогою оператора new і відповідного конструктора, наприклад:

String str1 = "Hello world!";
String str2 = new String("Hello world!");

У класі String визначено наступні методи для роботи з рядками, які представлено у зведеній таблиці 8.2.

***Таблиця 8.2 – Методи, визначені у класі* String**

Метод	Опис
String concat(String s)	Виконує злиття або конкатенацію рядків
boolean equals(Object ob)	Виконує порівняння рядків з врахуванням їх регістру. Якщо рядки рівні між собою повертається значення true
boolean equalsIgnoreCase(String s)	Виконує порівняння рядків без врахування їх регістру. Якщо рядки рівні між собою повертається значення true
int compareTo(String s)	Виконує лексикографічне порівняння рядків з врахуванням їх регістру. Якщо рядки рівні між собою повертається значення 0
int compareToIgnoreCase(String s)	Виконує лексикографічне порівняння рядків без врахування їх регістру. Якщо рядки рівні між собою повертається значення 0
boolean contentEquals(StringBuffer ob)	Виконує порівняння рядка і переданого об'єкта типу StringBuffer. Якщо рядки рівні між собою повертається значення true
String substring(int n, int m)	Витягує з рядка підрядок довжиною від позиції n до позиції m. Нумерація символів в рядку починається з нуля
String substring(int n)	Витягує з рядка підрядок, починаючи з позиції n
int length()	Дозволяє визначити довжину рядка
int indexOf(char ch)	Визначає позицію символу у рядку
static String valueOf(base type value)	Виконує перетворення змінної базового типу до рядка
String toUpperCase()	Виконує приведення усіх символів рядка до верхнього регістру

String toLowerCase()	Виконує приведення усіх символів рядка до нижнього регістру
String replace(char c1, char c2)	Виконує заміну в рядку усіх входжень c1 на входження c2
String intern()	Заносить рядок у «пул» літералів і повертає його посилання на об'єкт типу String
String trim()	Видаляє усі пропуски на початку і в кінці рядка
char charAt(int position)	Повертає символ із вказаної позиції. Нумерація символів в рядку починається з нуля
boolean isEmpty()	Повертає значення true, якщо довжина рядка дорівнює 0
byte[] getBytes()	Поміщає символи з рядка в масив типу byte
char[] getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)	Копіює символи з рядка в масив типу char, починаючи із позиції srcBegin та закінчуючи srcEnd. Змінна dstBegin відповідає за позицію з якої починається формування нового масиву
static String format(String format, Object args)	Генерує форматований рядок, отриманий з використанням відповідного формату
String[] split(String regex)	Виконує пошук входження в рядок заданого регулярного виразу і поділ початкового рядка на відповідний масив рядків

У всіх випадках, коли викликаються методи, що призводять до зміни рядка, створюється новий об'єкт типу String.

У наступному прикладі демонструється перетворення масиву символів і цілого числа в об'єкти типу String з використанням розглянутих вище методів.

```
//Використання методів класу String
public class Main {
  static int i;

  public static void main(String[] args) {
        char s[] = {'J', 'a', 'v', 'a'}; // масив

        // В коментарях містяться проміжні результати виконання програми
        String str = new String(s); // str = "Java"

        if (!str.isEmpty()) {
              i = str.length(); // i = 4
              str = str.toUpperCase(); // st r= "JAVA"
              String num = String.valueOf(6); // num = "6"
              num = str.concat("-" + num); // num = "JAVA-6"
              char ch = str.charAt(2); // ch = 'V'
              i = str.lastIndexOf('A'); // i = 3
              num = num.replace("6", "SE"); // num = "JAVA-SE"
              str.substring(0, 4).toLowerCase(); // java
```

```
                str = num + "-6"; // str = "JAVA-SE-6"
                String[] arr = str.split("-");

                for (String ss : arr)
                        System.out.println(ss);
        } else {
                System.out.println("String is empty!");
        }
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
JAVA
SE
6
```

Зберегти зміни в об'єкті класу String можна тільки за допомогою оператора присвоєння, тобто шляхом встановлення посилання на новий об'єкт.

У наступному прикладі розглядаються особливості зберігання та ідентифікації об'єктів на прикладі виклику методу equals(), що порівнює рядок типу String із вказаним об'єктом і методу hashCode(), який обчислює хеш-код переданого об'єкта.

```
// Порівняння посилань та об'єктів
public class Main {
  public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");

        System.out.println(s1 + "==" + s2 + " : " + (s1 == s2));
        System.out.println(s1 + "==" + s3 + " : " + (s1 == s3));

        System.out.println(s1 + " equals " + s2 + " : " +
  s1.equals(s2));
        System.out.println(s1 + " equals " + s3 + " : " +
  s1.equals(s3));

        System.out.println(s1.hashCode());
        System.out.println(s2.hashCode());
        System.out.println(s3.hashCode());
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Java==Java : true
Java==Java : false
Java equals Java : true
Java equals Java : true
2301506
2301506
2301506
```

Незважаючи на те, що однакові за значенням рядкові об'єкти розташовані в різних ділянках пам'яті, значення їх хеш-кодів співпадають. Оскільки у Java всі посилання зберігаються в стеку, а об'єкти – в купі, то при створенні об'єкту s2 спочатку створюється посилання, а потім це посилання зв’язується з відповідним об’єктом. Тому s2 асоціюється із вже існуючим літералом, оскільки об'єкт s1 уже зв’язаний із посилання на цей літерал. При створенні об’єкту s3 відбувається виклик конструктора, тобто виділення пам'яті відбувається раніше за його ініціалізацію, і тому в купу поміщається новий об'єкт.

У мові Java можна об’єднувати рядки із даними інших типів. Для цього використовується оператор об’єднання або конкатенації «+». Під час операції об’єднання усі типи даних приводяться автоматично до рядків. Компілятор перетворює операнди в їх рядкові еквіваленти, в той час як інші операнди оператора «+» є екземплярами класу String.

Проте, під час такого об’єднання необхідно враховувати один досить важливий момент. Розглянемо фрагмент програми в якому відбувається об’єднання декількох рядків:

```
String str = "Десять: " + 4 + 6;
System.out.println(str);
```

Запустивши програму на виконання отримаємо наступний результат:

```
Десять: 46
```

Такий результат є досить логічним, оскільки пріоритети операторів забезпечують спочатку об’єднання рядка "Десять: " з рядковим еквівалентом числа 4, а потім результат об’єднується з рядковим еквівалентом числа 6.

Для того, щоб отримати правильний результат необхідно змінити пріоритет виконання операцій, шляхом вкладення виразу «4 + 6» в круглі дужки.

```
String str = "Десять: " + (4 + 6);
System.out.println(str);
```

Запустивши програму на виконання отримаємо наступний результат:

```
Десять: 10
```

Під час операції об’єднання Java перетворює дані в рядкове представлення викликаючи для цього метод valueOf(), який визначено у класі String. Цей метод перевантажений для всіх елементарних типів даних і для типу Object. У випадку елементарних типів даних, метод повертає читабельний для людини еквівалент значення з яким він був викликаний.

Для об’єктів метод valueOf() викликає метод toString() який реалізується усіма об’єктами, оскільки він визначений у класі Object. Проте, реалізація методу toString() по замовчуванню досить рідко буває корисною. Тому розглянемо приклад перевизначення цього методу для класу Box.

```
// Перевизначення методу toString() у класі Box
class Box {
  double width;
  double height;
  double depth;
```

```
  Box (double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
  }

  public String toString() {
        return "Розміри " + width + " на " + height + " на " + depth;
  }
}

public class Main {
  public static void main(String[] args) {
        Box mybox = new Box(10, 20, 15);
        System.out.println(mybox);
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Розміри 10.0 на 20.0 на 15.0
```

Щоб реалізувати метод toString() необхідно повернути об’єкт класу String, який містить читабельний для людини рядок, що адекватно описує поведінку об’єкта класу.

Класи StringBuilder *та* StringBuffer

Класи StringBuilder та StringBuffer є «близнюками» і по своїй функціональності близькі до класу String, але, на відміну від останнього, вміст і розміри об'єктів класів StringBuilder та StringBuffer можна змінювати.

Основною і єдиною відмінністю класу StringBuilder від StringBuffer є потокобезпечність останнього. У версії Java 1.5.0 був доданий не потокобезпечний (тобто, швидший в обробці) клас StringBuilder, який слід застосовувати, у випадках, якщо не існує ймовірності використання об'єкту в конкуруючих між собою потоках.

За допомогою відповідних методів і конструкторів об'єкти класів StringBuffer, StringBuilder і String можна перетворювати один в одного. Конструктори класу StringBuffer та StringBuilder можуть приймати як параметр об'єкт класу String так і вміст буфера. Об'єкти цього класу можна перетворити в об'єкт класу String за допомогою методу toString() або за допомогою конструктора класу String.

Методи, що визначені у класах StringBuffer та StringBuilder та які заслуговують на увагу представлено у таблиці 8.3.

***Таблиця 8.3 – Методи, визначені у класах* StringBuffer *та* StringBuilder**

Метод	Опис
void setLength(int n)	Дозволяє встановити розмір буфера

void ensureCapacity(int minimum)	Встановлює гарантований мінімальний розмір буфера
int capacity()	Повертає поточний розмір буфера
StringBuffer append(параметри)	Виконує додавання до вмісту об'єкта строкового типу аргумента, який може бути символом, значенням базового типу, масивом або рядком
StringBuffer insert(параметри, int index)	Дозволяє вставити символ, об'єкт або рядок у вказану позицію
StringBuffer deleteCharAt(int index)	Видаляє символ із вказаної позиції
StringBuffer delete(int start, int end)	Видаляє підрядок, що починається з позиції start та закінчується позицією end
StringBuffer reverse()	Виконує дзеркальне відображення рядкового об'єкта

У класах StringBuffer та StringBuilder визначено також методи аналогічні методам класу String.

При створенні об'єкту класу StringBuffer конструктор за замовчуванням автоматично резервує деякий об'єм пам'яті (16 символів), що в подальшому дозволяє швидко змінювати вміст об'єкту, залишаючись в межах ділянки пам'яті, виділеної під об'єкт. Розмір зарезервованої пам'яті при необхідності можна вказати в конструкторі. Якщо довжина рядка StringBuffer після зміни перевищує його розмір, то об’єм пам’яті автоматично збільшується.

Якщо метод, що викликається об'єктом StringBuffer, модифікує його вміст, то це не призводить до створення нового об'єкту, а виконується модифікація поточного об'єкту класу StringBuffer.

```
// Модифікація об'єкта класу StringBuffer
public class Main {
  public static void changeStr(StringBuffer s) {
        s.append(" world!");
  }

  public static void main(String[] args) {
        StringBuffer str = new StringBuffer("Hello");
        changeStr(str);
        System.out.println(str);
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
Hello world!
```

Об'єкт класу StringBuffer передається у метод changeStr() по посиланню, тому усі зміни пов’язані із об'єктом зберігаються.

У класі StringBuffer не реалізовано перевизначених методів equals() і hashCode(), що не дозволяє порівняти вміст двох об'єктів.

Форматування рядків

Для створення форматованого тексту у мові Java використовується клас Formatter, який входить у пакет java.util. Цей клас дозволяє виводити числа, рядки, час і дату у будь-якій зручній для кінцевого користувача формі.

У класі Formatter оголошений метод format(), який виконує перетворення переданого в нього параметра в рядок заданого формату і зберігає його в об'єкті типу Formatter. Аналогічний метод оголошений у класах PrintStream та PrintWriter. Крім того, у цих класах оголошено метод printf() з параметрами ідентичними параметрам методу format(), який виконує форматоване виведення в потік, тоді як метод format() зберігає зміни в об'єкті типу Formatter.

Клас Formatter перетворює двійкову форму представлення даних у форматований текст. Він зберігає форматований текст у буфері, вміст якого можна отримати у будь-який необхідний момент. Також існує можливість збереження буфера класу Formatter у файл.

У класі Formatter визначено більше десяти конструкторів. Основні із них приведено у зведеній таблиці 8.4

***Таблиця 8.4 – Конструктори класу* Formatter**

Formatter()
Formatter(Appendable buf)
Formatter(Appendable buf, Locale loc)
Formatter(String filename) throws FileNotFoundException
Formatter(String filename, String charset)
throws FileNotFoundException, UnsupportedEncodingException
Formatter(File outF) throws FileNotFoundException
Formatter(OutStream outStrm)

У наведених конструкторах параметр buf задає буфер для форматованого виведення. Якщо параметр buf рівний null, клас Formatter автоматично створює об'єкт типу StringBuilder для зберігання форматованого вмісту. Параметр loc визначає регіональні та мовні налаштування. Якщо ніяких налаштувань не задано, використовуються налаштування по замовчуванню. Параметр filename задає ім'я файлу, в який буде збережено форматований текст. Параметр charset визначає кодування. Якщо воно не задано, то використовується кодування, встановлене по замовчуванню. Параметр outF містить посилання на відкритий файл, в якому зберігається форматований текст. Через параметр outStrm передається посилання на потік виведення, який отримуватиме відформатовані дані. Якщо використовується файл, вихідні дані записуються у файл.

Основні методи визначені у класі Formatter представлено у зведеній таблиці 8.5

***Таблиця 8.5 – Методи, визначені у класі* Formatter**

Метод	Опис
Formatter format(String fmtString, Object...args)	Виконує форматування аргументів, що передаються через параметр змінної довжини args, відповідно до специфікаторів формату, що визначені в fmtString.
Formatter format(Locale loc, String fmtString, Object...args)	Виконує форматування аргументів, що передаються через параметр змінної довжини args, відповідно до специфікаторів формату, що визначені в fmtString. При форматуванні використовуються регіональні установки, задані в параметрі loc.
Locale locale()	Повертає регіональні установки об'єкту, що форматується
Appendable out()	Повертає посилання на базовий об'єкт-приймач для вихідних даних
void flush()	Переносить інформацію з буфера форматування і виконує запис вихідних даних, що знаходяться у буфері у вказане місце. Метод найчастіше використовується об'єктом класу Formatter, пов'язаним з файлом
void close()	Закриває об'єкт класу Formatter, що призводить до звільнення ресурсів, використовуваних об'єктом. Після закриття об'єкту типу Formatter він не може використовуватися повторно. Спроба використати закритий об'єкт призводить до генерації винятку FormatterClosedEception
String toString()	Повертає об'єкт типу String, що в якому міститься відформатований текст

Під час форматування використовуються специфікатори формату представлені у таблиці 8.6.

Таблиця 8.6 – Специфікатори формату

Специфікатор формату	Виконуване форматування
%a	Шістнадцяткове значення з плаваючою крапкою
%b	Логічне значення аргумента
%c	Символьне представлення аргумента
%d	Десяткове ціле значення аргумента
%h	Хеш-код аргумента
%e	Експоненціальне представлення аргумента
%f	Десяткове значення з плаваючою крапкою
%g	Вибирає коротше представлення з двох: %e або %f
%o	Вісімкове ціле значення аргумента

%n	Вставка символу нового рядка
%s	Рядкове представлення аргумента
%t	Час і дата
%x	Шістнадцяткове ціле значення аргумента
%%	Вставка знаку %

Також можливе використання специфікаторів із прописними літерами: (%A еквівалентно %a). Форматування за допомогою них забезпечує перевід символів у верхній регістр.

```
// Форматування рядків та чисел за допомогою методу format()
import java.util.Formatter;

public class Main {
  public static void main(String[] args){

        // Створення об'єкта класу Formatter
        Formatter txt = new Formatter();
        Formatter num = new Formatter();

        // Форматування тексту
        txt.format("This %s is about %n%S %c", "book","java",'6');
        System.out.println(txt);

        // Форматування чисел
        num.format("Hex: %x, Octal:  %o", 100, 100);
        System.out.println(num);
        num = new Formatter();
        num.format("%a", 100.001);
        System.out.println(num);
        num = new Formatter();
        for (double i = 1000; i < 1.0e+10; i *= 100) {
                num.format("%g ", i);
        System.out.println(num);
        }
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
This book is about
JAVA 6
Hex: 64, Octal:  144
0x1.90010624dd2f2p6
1000,00
1000,00 100000
1000,00 100000 1,00000e+07
1000,00 100000 1,00000e+07 1,00000e+09
```

У табл. 8.7 приведено основні специфікатори формату часу і дати.

Таблиця 8.7 – Специфікатори формату часу і дати

Специфікатор формату	Виконуване форматування
%tH	Година (00 - 23)
%tI	Година (1 - 12)
%tM	Хвилини як десяткове ціле (00 - 59)
%tS	Секунди як десяткове ціле (00 - 59)
%tL	Мілісекунди (000 - 999)
%tY	Рік в чотиризначному форматі(0000 - 9999)
%ty	Рік в двозначному форматі (00 - 99)
%tB	Повна назва місяця («January»)
%tb або %th	Коротка назва місяця («Jan»)
%tm	Місяць в двозначному форматі (1 - 12)
%tA	Повна назва дня тижня ("Monday")
%ta	Коротка назва дня тижня ("Mon")
%td	День в двозначному форматі (1 - 31)
%tR	Те ж саме що і «%tH:%tM»
%tT	Те ж саме що і «%tH:%tM:%tS»
%tr	Те ж саме що і «%tI:%tM:%tS %Tp» де %Tp = (AM або PM)
%tD	Те ж саме що і «%tm/%td/%ty»
%tF	Те ж саме що і «%tY - %tm - %td»
%tc	Те ж що і «%ta %tb %td %tT %tZ %tY»

Усі специфікатори для форматування дати і часу можуть бути використані тільки для типів long, Long, Calendar, Date.

```
// Форматування дати та часу за допомогою методу format()
import java.util.*;

public class Main {
  public static void main(String args[]) {

        Formatter f = new Formatter();
        Calendar cal = Calendar.getInstance();

        // Вивід у 12-часовому форматі
        f.format("%tr", cal);
        System.out.println(f);
```

```
        // Повноформатний вивід дати та часу
        f = new Formatter();
        f.format("%tc", cal);
        System.out.println(f);

        // Вивід поточної години та хвилин
        f = new Formatter();
        f.format("%tl:%tM", cal, cal);
        System.out.println(f);

        // Вивід поточного місяця
        f = new Formatter();
        f.format("%tB %tb %tm", cal, cal, cal);
        System.out.println(f);
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
04:04:45 PM
пт лют 14 16:04:45 EET 2014
4:04
лютого лют 02
```

Специфікатор точності застосовується тільки в специфікаторах формату %f, %e та %g для даних з плаваючою крапкою і в специфікаторі %s – для рядків. Він задає кількість десяткових знаків, що виводяться, або символів. Наприклад, специфікатор %10.4f виводить число з мінімальною шириною поля 10 символів і з чотирма десятковими знаками. Прийнята за замовчуванням точність дорівнює шести десятковим знакам.

Застосовний до рядків специфікатор точності задає максимальну довжину поля виведення. Наприклад, специфікатор %5.7s виводить рядок довжиною не менше п'яти і не більше семи символів. Якщо рядок довший, то кінцеві символи відкидаються.

```
// Використання прапорців форматування
import java.util.*;

public class Main {
  public static void main(String[] args) {
        Formatter f = new Formatter();

        // Вирівнювання по правій стороні
        f.format("|%10.2f|", 123.123);
        System.out.println(f);

        // Вирівнювання по лівій стороні та використання прапорця '-'
        f = new Formatter();
        f.format("|%-10.2f|", 123.123);
        System.out.println(f);

        // Використання прапорця ' ' та '('
```

```
        f = new Formatter();
        f.format("% (d", -100);
        System.out.println(f);

        // Використання прапорця ','
        f = new Formatter();
        f.format("%,.2f", 123456789.34);
        System.out.println(f);

        // Задання точності представлення для чисел
        f = new Formatter();
        f.format("%.4f", 1111.1111111);
        System.out.println(f);

        // Задання точності представлення для рядків
        f = new Formatter();
        f.format("%.16s", "Now I know class java.util.Formatter");
        System.out.println(f);
  }
}
```

Запустивши програму на виконання отримаємо наступний результат:

```
|    123,12|
|123,12    |
(100)
123 456 789,34
1111,1111
Now I know class
```

У класу Formatter є одна корисна властивість, яка дозволяє задавати аргумент, до якого слід застосувати конкретний специфікатор формату. За замовчуванням відповідність між специфікаторами і аргументами, на які вони впливають, встановлюється відповідно до порядку їх розміщення, зліва направо. Це означає, що перший специфікатор формату відповідає першому аргументу, другий специфікатор – другому аргументу і т. д. Проте, використовуючи порядковий номер або індекс аргументу, можна вказати явну відповідність специфікатора формату аргумента.

Порядковий номер аргумента вказується за знаком % в специфікаторі формату і має наступний формат: N$. Символ N означає порядковий номер потрібного аргументу, нумерація аргументів починається з одиниці.

Висновки

Рядки в java мають двояку природу – з однієї сторони вони поводять себе як елементарні типи даних, а з іншої – як об’єкти визначеного класу. У мові Java для роботи з рядками визначено аж три класи, а саме String, StringBuilder та StringBuffer, кожний з яких вносить свої особливості, які були розглянуті вище.

Проте самі по собі рядки є лише звичайним не відформатованим текстом, який представляється кінцевому користувачу досить часто в не дуже зручній формі. Тому в мові Java також визначено клас Formatter в якому реалізовано методи, що дозволяють виконувати форматування будь-якого рядка та представляти його у зручній для користувача формі. Особливо корисним є цей

клас у випадку роботи із рядками, що представляють числа, час та дату.

Контрольні запитання

1. Чим виступає рядкова змінна у мові Java?
2. Назвіть класи, які використовуються для роботи з рядками.
3. Назвіть причину по якій не можна створити власні породжені класи з властивостями рядків.
4. Яка специфіка використання класу String?
5. Перелічіть способи за допомогою яких можна створити рядкову змінну.
6. Який оператор у мові Java дозволяє виконати об'єднання рядків?
7. У чому полягає різниця між методами valueOf() та toString()?
8. Яка специфіка використання класів StringBuilder і StringBuffer?
9. Назвіть відмінність між класами StringBuilder і StringBuffer.
10. Який клас у мові Java використовується для форматування рядків?
11. Назвіть метод, який виконує перетворення переданого в нього параметра в рядок заданого формату.
12. У чому полягає відмінність між методами format() та printf()?
13. Яке призначення специфікаторів формату?
14. Яким чином встановлюється відповідність між специфікаторами формату та аргументами на які вони впливають?
15. Назвіть сферу застосування регулярних виразів у мові Java.
16. Які класи використовуються для формування регулярних виразів?
17. У чому полягає відмінність між класами Pattern та Matcher?
18. Що таке метасимволи та яка сфера їх застосування?
19. У яких випадках використовуються символьні класи?
20. Яка роль квадратних та круглих дужок у регулярних виразах?
21. Назвіть причину використання квантифікаторів у регулярних виразах.